



MERRIMACK COLLEGE

CSC 6302 - Database Principles

Week 6 - Amanda Menier

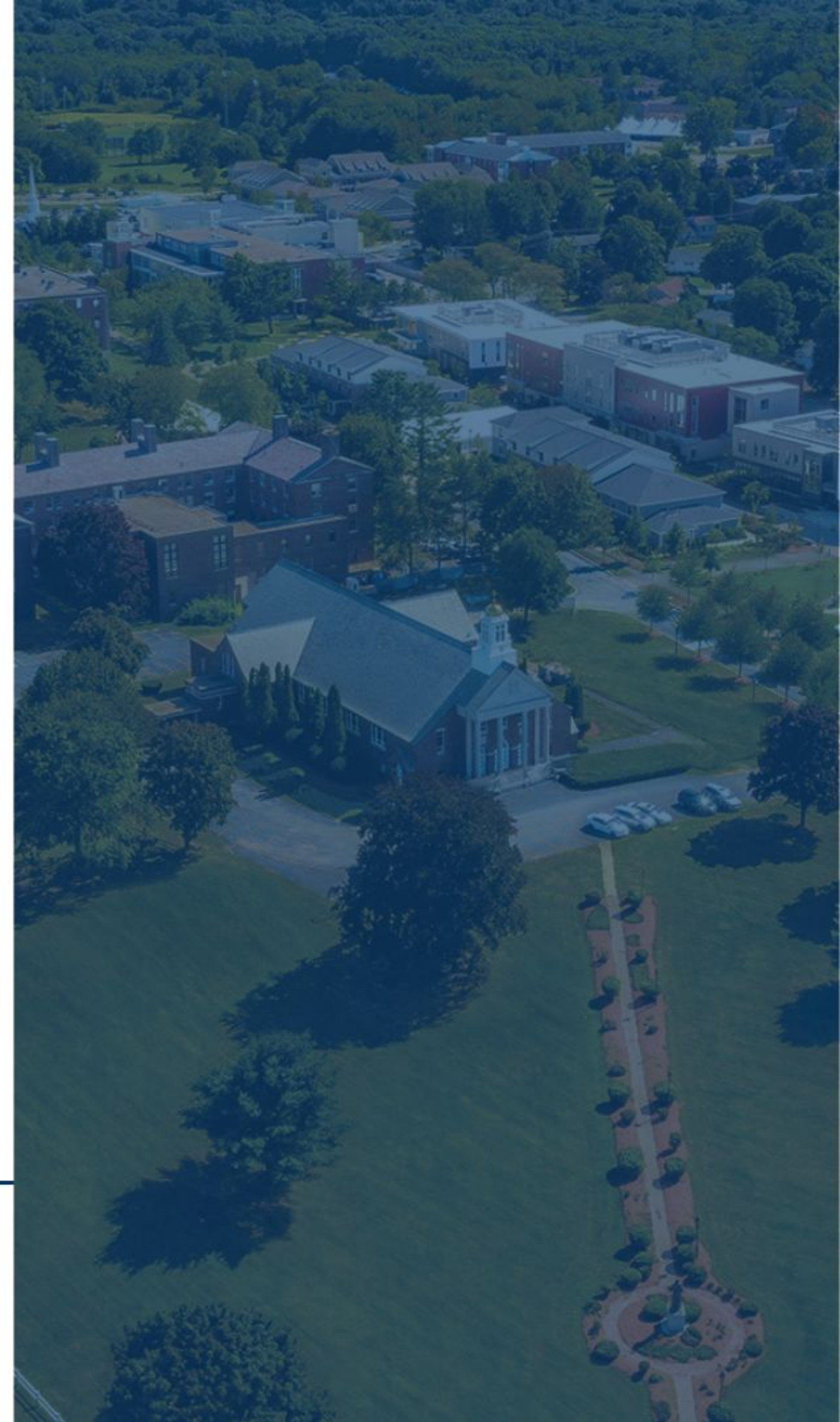
Agenda

Discussion Topics:

- Project 5 Feedback
- Storage
- RAID
- Authorization
- Authentication



MERRIMACK COLLEGE



Project 5 Feedback

- 1) Login screen incomplete - missing port
- 2) Missing drop downs or date and time pickers
- 3) Double booking: One way to implement:
 - a) Get the time and duration of the new trip.
 - b) Get trips for that passenger and vessel.
 - c) See whether the start or end of the new trip fall between the start and end of existing trips
 - i) If there is a conflict, don't add the trip
 - ii) Otherwise, add the trip
- 4) The password you hard-coded is for your MySQL server instance. Mine is different, so to run on my machine, you need to save and use my credentials

Classification of Physical Storage Media

Classification of Physical Storage Media

volatile storage: loses contents when power is switched off

non-volatile storage:

Contents persist even when power is switched off.

Includes secondary and tertiary storage, as well as batter-backed up main-memory.

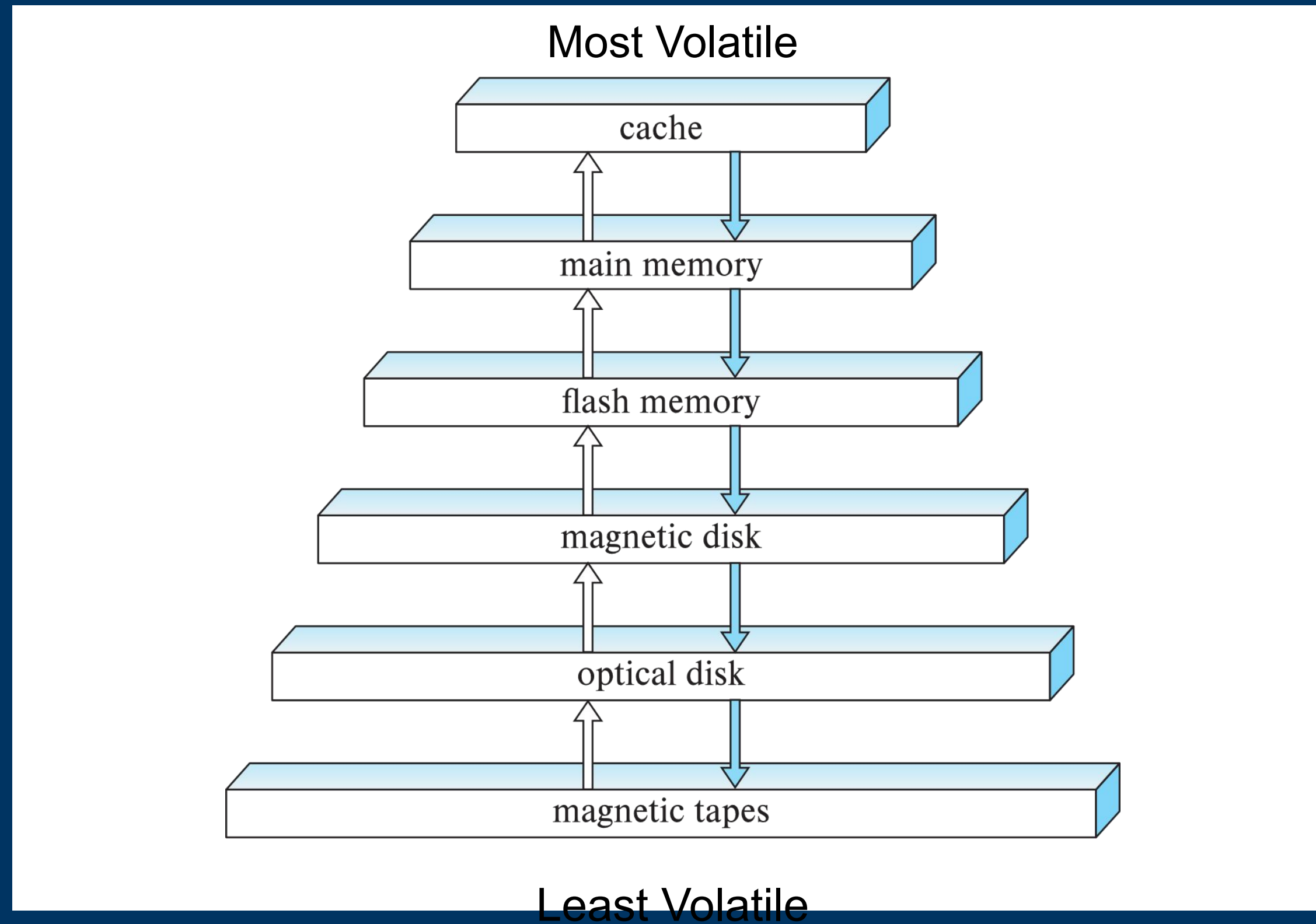
Factors affecting choice of storage media include

Speed with which data can be accessed

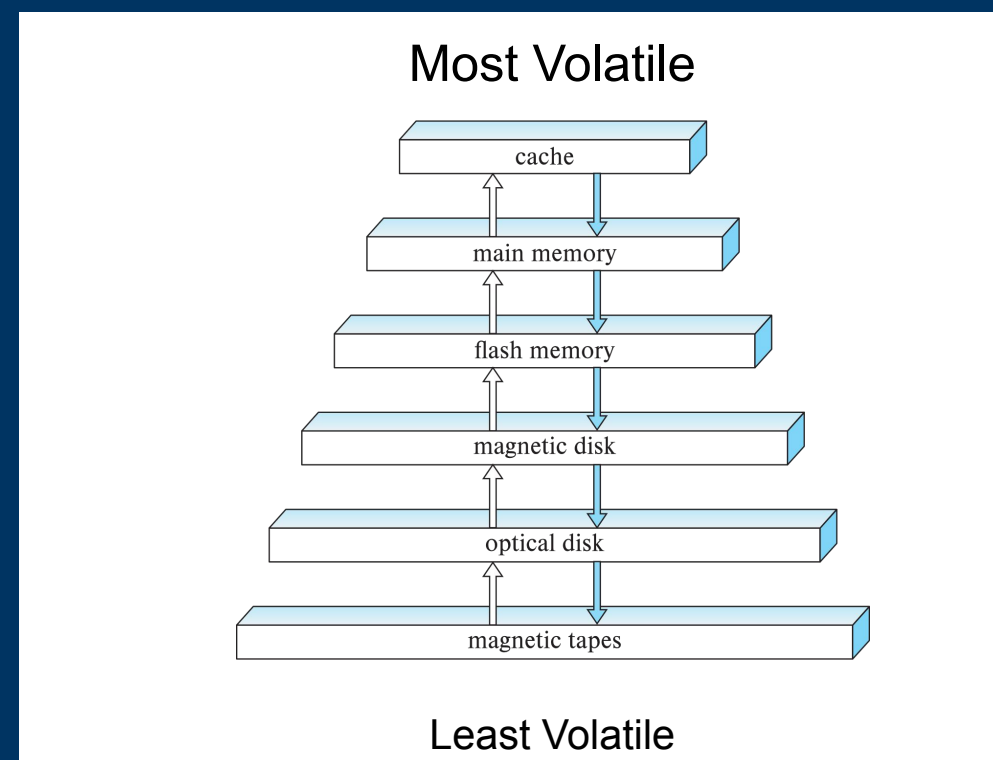
Cost per unit of data

Reliability

Storage Hierarchy



Storage Hierarchy



primary storage: Fastest media but volatile (cache, main memory).

secondary storage: next level in hierarchy, non-volatile, moderately fast access time

Also called **on-line storage**

E.g., flash memory, magnetic disks

tertiary storage: lowest level in hierarchy, non-volatile, slow access time

also called **off-line storage** and used for **archival storage**

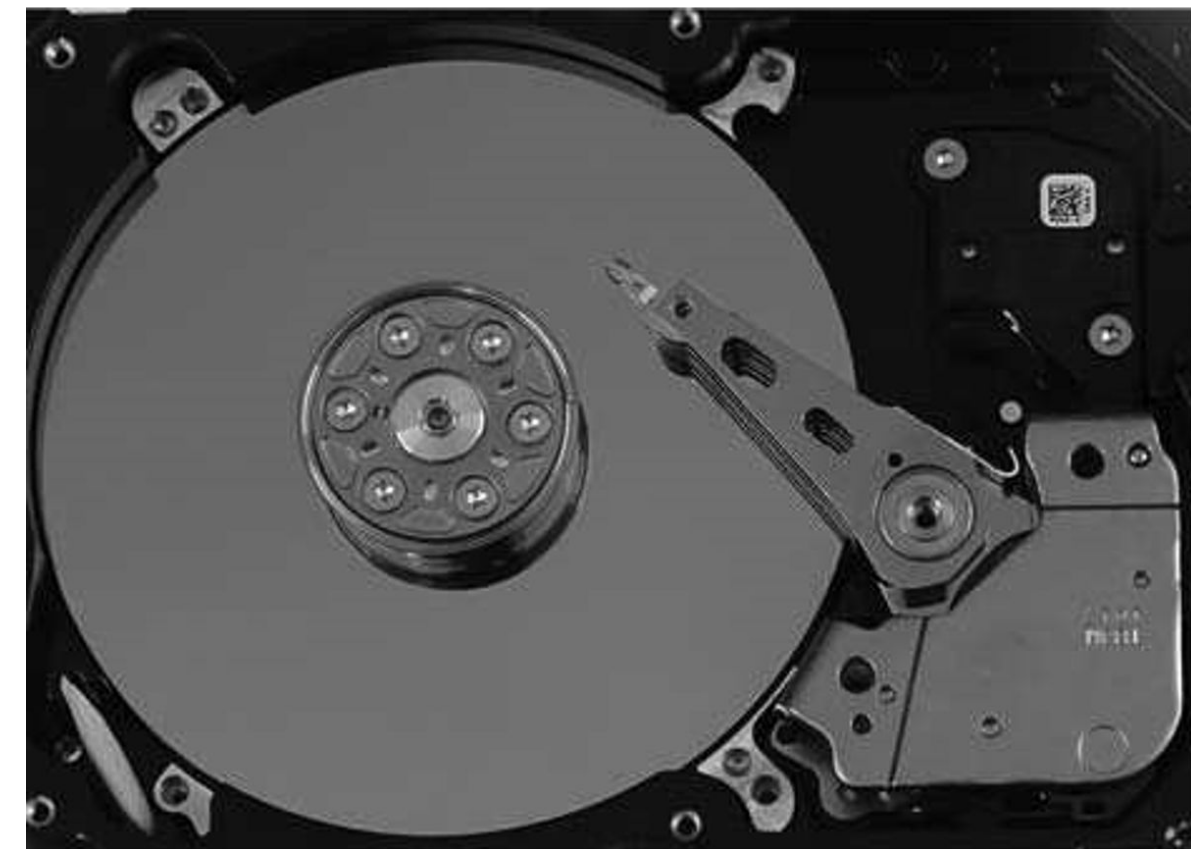
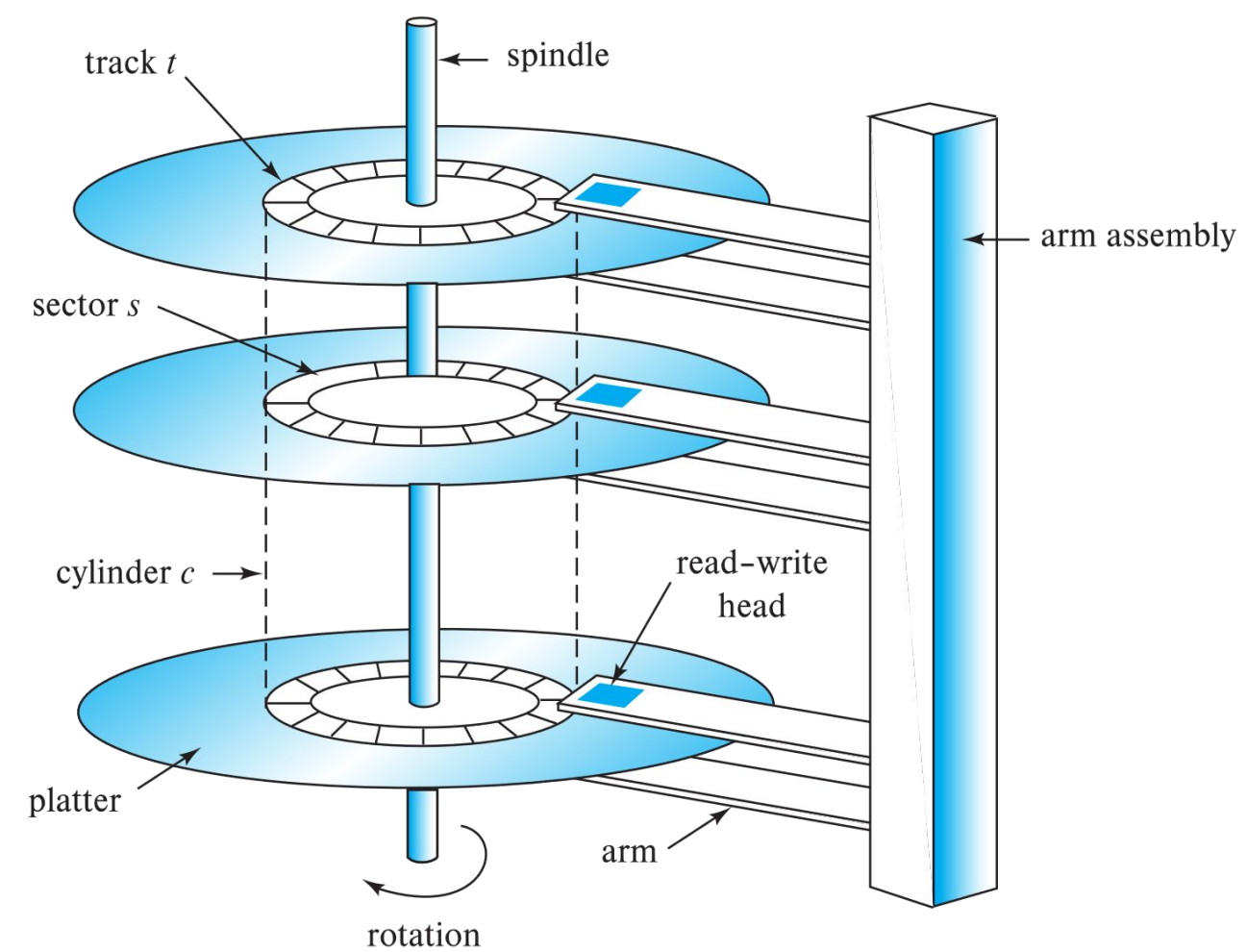
e.g., magnetic tape, optical storage

Magnetic tape

Sequential access, 1 to 12 TB capacity

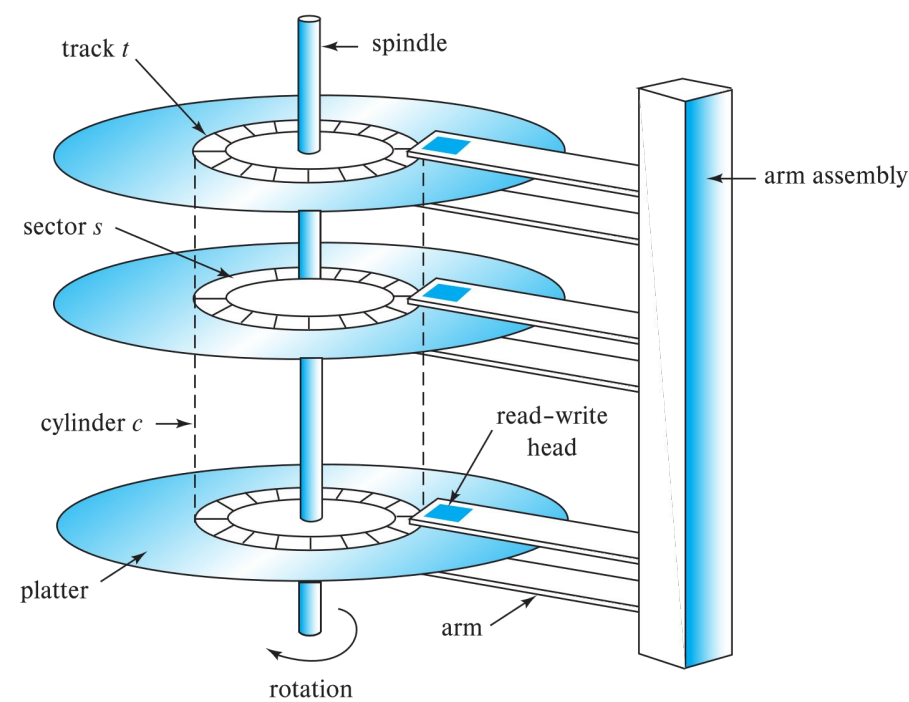
A few drives with many tapes

Magnetic Hard Disk Mechanism



Schematic diagram of magnetic disk drive Photo of magnetic disk drive

Magnetic Hard Disk Mechanism



Schematic diagram of magnetic disk drive

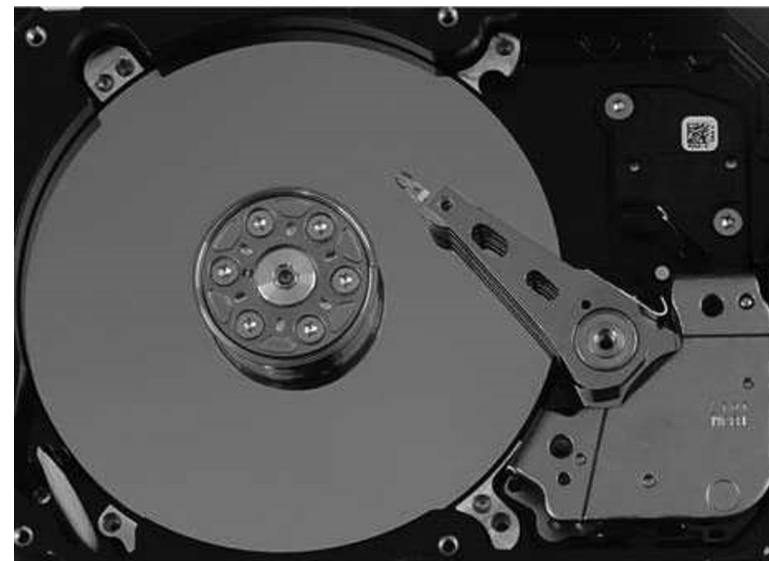


Photo of magnetic disk drive

Read-write head

Surface of platter divided into circular **tracks**

Over 50K-100K tracks per platter on typical hard disks

Each track is divided into **sectors**.

A sector is the smallest unit of data that can be read or written.

Sector size typically 512 bytes

Typical sectors per track: 500 to 1000 (on inner tracks) to 1000 to 2000 (on outer tracks)

To read/write a sector

disk arm swings to position head on right track

platter spins continually; data is read/written as sector passes under head

Head-disk assemblies

multiple disk platters on a single spindle (1 to 5 usually)

one head per platter, mounted on a common arm.

Disk Controller

Disk controller – interfaces between the computer system and the disk drive hardware.

Accepts high-level commands to read or write a sector, initiates actions such as moving the disk arm to the right track and actually reading or writing the data

Computes and attaches **checksums** to each sector to verify that data is read back correctly

If data is corrupted, with very high probability stored checksum won't match recomputed checksum

Ensures successful writing by reading back sector after writing it

Performs **remapping of bad sectors**

Performance Measures of Disks

Access time – the time it takes from when a read or write request is issued to when data transfer begins.

Consists of:

Seek time – time it takes to reposition the arm over the correct track.

Average seek time is $1/2$ the worst case seek time.

Would be $1/3$ if all tracks had the same number of sectors, and we ignore the time to start and stop arm movement

4 to 10 milliseconds on typical disks

Rotational latency – time it takes for the sector to be accessed to appear under the head.

4 to 11 milliseconds on typical disks (5400 to 15000 r.p.m.)

Average latency is $1/2$ of the above latency.

Overall latency is 5 to 20 ms depending on disk model

Data-transfer rate – the rate at which data can be retrieved from or stored to the disk.

Performance Measures (Cont.)

Disk block is a logical unit for storage allocation and retrieval

4 to 16 kilobytes typically

Smaller blocks: more transfers from disk

Larger blocks: more space wasted due to partially filled blocks

Sequential access pattern

Successive requests are for successive disk blocks

Disk seek required only for first block

Random access pattern

Successive requests are for blocks that can be anywhere on disk

Each access requires a seek

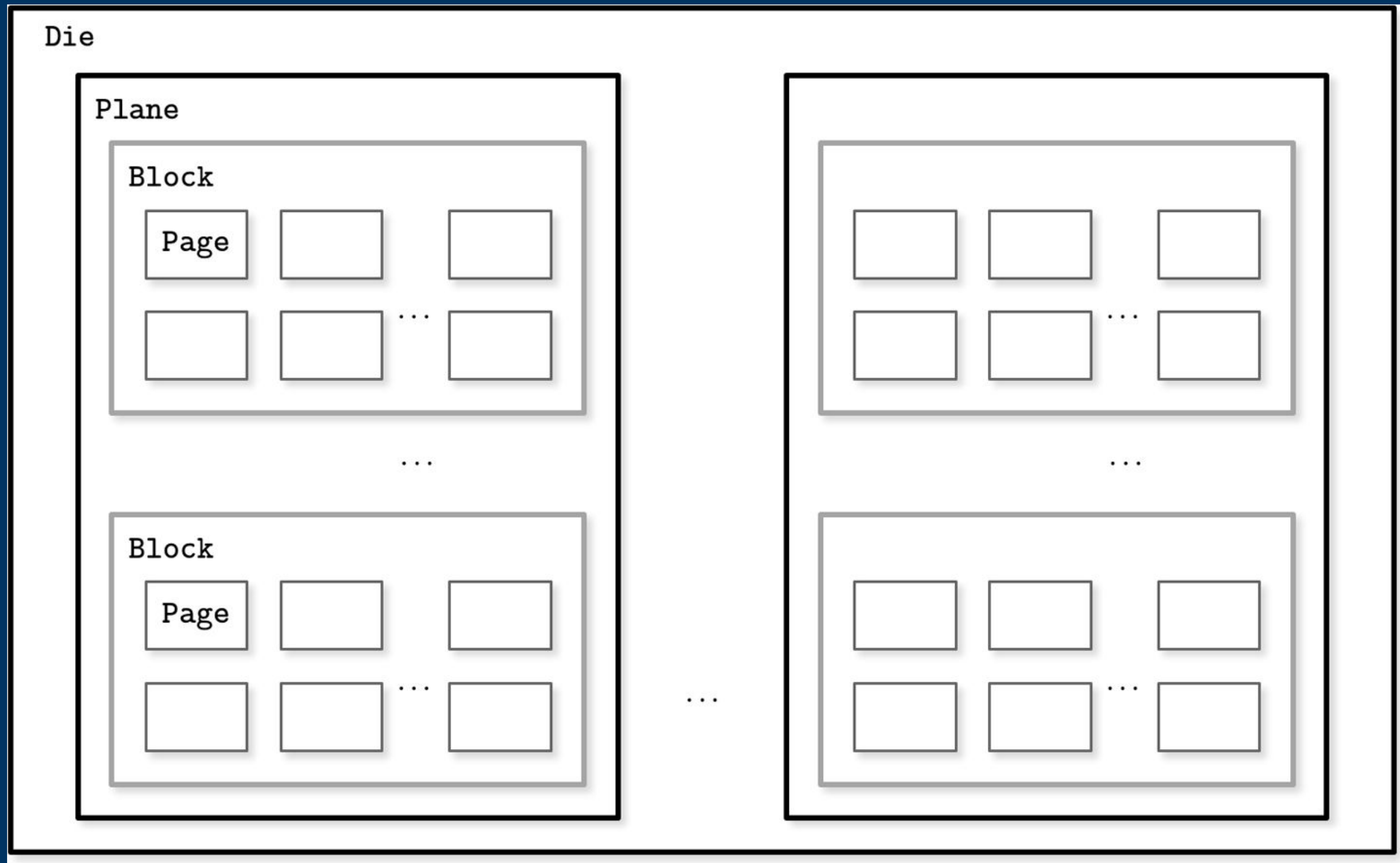
Transfer rates are low since a lot of time is wasted in seeks

I/O operations per second (IOPS)

Number of random block reads that a disk can support per second

50 to 200 IOPS on current generation magnetic disks

SSD



Performance Measures (Cont.)

Mean time to failure (MTTF) – the average time the disk is expected to run continuously without any failure.

$$\text{eg. MTTF} = \frac{500,000 \text{ hours}}{5 \text{ failures}} = 100,000 \text{ hours}$$

It's typically based on manufacturer data and is a statistical estimate

Not a guaranteed lifespan for individual units.

Performance Measures (Cont.)

MTTF for Multiple Disks - The chance of failure increases with more disks!

$$\text{System MTTF} = \frac{\text{Single Disk MTTF}}{N \text{ Disks}}$$

As N increases the MTTF for the system goes down

For a single disk: the probability of failure is relatively low because it has a high MTTF

For **N** disks: the combined probability of failure is much higher because **any** one of those disks could fail

Performance Measures (Cont.)

MTTF for Multiple Disks - The chance of failure increases with more disks!

$$\text{eg. } \text{MTTF} = \frac{100,000 \text{ hours}}{100 \text{ Disks}} = 1,000$$

This means that, for the system as a whole, the average time before **any disk fails** is only 1,000 hours, which is approximately **41 days**

For **N** disks: the combined probability of failure is much higher because **any** one of those disks could fail

RAID

Redundant Arrays of Independent Disks

A sophisticated data storage virtualization technology that combines multiple physical disk drive components into one or more logical units

Increase the data integrity, availability, and performance of a computer's storage architecture through the use of multiple disks

Improves system performance by spreading data across several disks and enhances data security by duplicating the data across multiple drives

Key Benefits:

Increased Data Reliability and Redundancy

Enhances Speed

Higher Data Capacity

Improvement of Reliability via Redundancy

Redundancy – store extra information that can be used to rebuild information lost in a disk failure

E.g., **Mirroring** (or **shadowing**)

- Duplicate every disk. Logical disk consists of two physical disks.

- Every write is carried out on both disks

- Reads can take place from either disk

- If one disk in a pair fails, data still available in the other

- Data loss would occur only if a disk fails, and its mirror disk also fails before the system is repaired

- Probability of combined event is very small

- Except for dependent failure modes such as fire or building collapse or electrical power surges

Mean time to data loss depends on mean time to failure,
and **mean time to repair**

Mean Time to Data Loss

In a Mirrored Disk Setup:

MTTF = 100,000 hours (Mean Time to Failure of each disk)

MTTR = 10 hours (Mean Time to Repair)

$$\text{MTTDL} = \frac{\text{MTTF}^2}{2 \times \text{MTTR}}$$

$$\text{MTTDL} = \frac{(100,000)^2}{2 \times 10}$$

$$\text{MTTDL} = \frac{10,000,000,000}{20} = 500,000,000 \text{ hours}$$

This means the Mean Time to Data Loss is 500 million hours, or approximately 57,000 years!!!

Improvement in Performance via Parallelism

Two main goals of parallelism in a disk system:

1. Load balance multiple small accesses to increase throughput
2. Parallelize large accesses to reduce response time.

Improve transfer rate by striping data across multiple disks.

Bit-level striping – split the bits of each byte across multiple disks

In an array of eight disks, write bit i of each byte to disk i .

Each access can read data at eight times the rate of a single disk.

But seek/access time worse than for a single disk

Bit level striping is not used much any more

Block-level striping – with n disks, block i of a file goes to disk $(i \bmod n) + 1$ Requests for different blocks can run in parallel if the blocks reside on different disks A request for a long sequence of blocks can utilize all disks in parallel

RAID Levels

Different RAID organizations, or RAID levels, have differing cost, performance and reliability characteristics

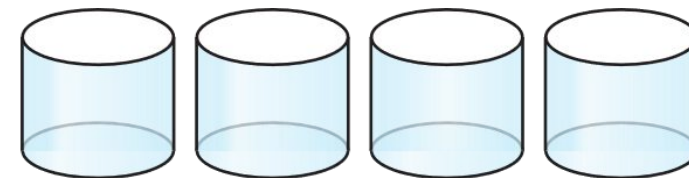
RAID Level 0: Block striping; non-redundant.

Used in high-performance applications where data loss is not critical.

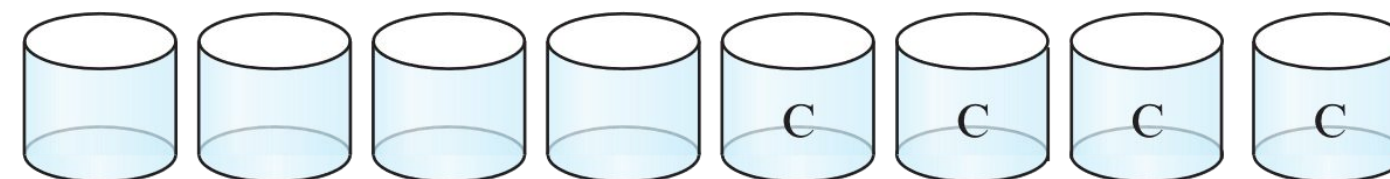
RAID Level 1: Mirrored disks with block striping

Offers best write performance.

Popular for applications such as storing log files in a database system.



(a) RAID 0: nonredundant striping



(b) RAID 1: mirrored disks

RAID Levels (Cont.)

Parity blocks: Parity block j stores XOR of bits from block j of each disk

When writing data to a block j , parity block j must also be computed and written to disk

Can be done by using old parity block, old value of current block and new value of current block (2 block reads + 2 block writes)

Or by recomputing the parity value using the new values of blocks corresponding to the parity block

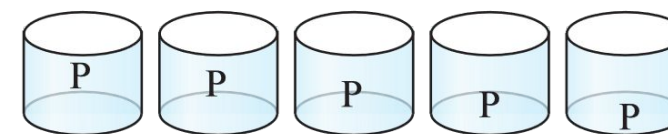
More efficient for writing large amounts of data sequentially

To recover data for a block, compute XOR of bits from all other blocks in the set including the parity block

RAID Levels (Cont.)

RAID Level 5: Block-Interleaved Distributed Parity; partitions data and parity among all $N + 1$ disks, rather than storing data in N disks and parity in 1 disk.

E.g., with 5 disks, parity block for n th set of blocks is stored on disk $(n \bmod 5) + 1$, with the data blocks stored on the other 4 disks.



(c) RAID 5: block-interleaved distributed parity

P0	0	1	2	3
4	P1	5	6	7
8	9	P2	10	11
12	13	14	P3	15
16	17	18	19	P4

RAID Levels (Cont.)

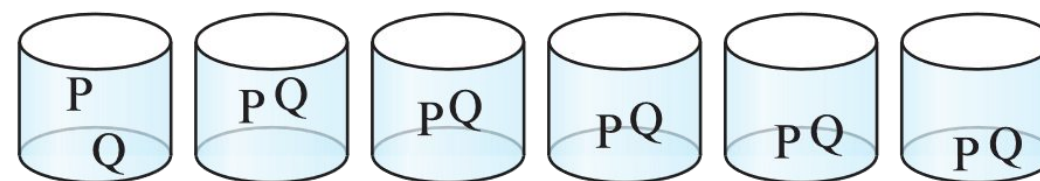
RAID Level 5 (Cont.)

Block writes occur in parallel if the blocks and their parity blocks are on different disks.

RAID Level 6: P+Q Redundancy scheme; similar to Level 5, but stores two error correction blocks (P, Q) instead of single parity block to guard against multiple disk failures.

Better reliability than Level 5 at a higher cost

Becoming more important as storage sizes increase



(d) RAID 6: P + Q redundancy

RAID Levels (Cont.)

Other levels (not used in practice):

RAID Level 2: Memory-Style Error-Correcting-Codes (ECC) with bit striping.

RAID Level 3: Bit-Interleaved Parity

RAID Level 4: Block-Interleaved Parity; uses block-level striping, and keeps a parity block on a separate ***parity disk*** for corresponding blocks from N other disks.

RAID 5 is better than RAID 4, since with RAID 4 with random writes, parity disk gets much higher write load than other disks and becomes a bottleneck

Choice of RAID Level

Factors in choosing RAID level

- Monetary cost

- Performance: Number of I/O operations per second, and bandwidth during normal operation

- Performance during failure

- Performance during rebuild of failed disk

 - Including time taken to rebuild failed disk

RAID 0 is used only when data safety is not important

- E.g., data can be recovered quickly from other sources

Choice of RAID Level (Cont.)

Level 1 provides much better write performance than level 5

Level 5 requires at least 2 block reads and 2 block writes to write a single block, whereas Level 1 only requires 2 block writes

Level 1 had higher storage cost than level 5

Level 5 is preferred for applications where writes are sequential and large (many blocks), and need large amounts of data storage

RAID 1 is preferred for applications with many random/small updates

Level 6 gives better data protection than RAID 5 since it can tolerate two disk (or disk block) failures

Increasing in importance since latent block failures on one disk, coupled with a failure of another disk can result in data loss with RAID 1 and RAID 5.

Hardware Issues

Software RAID: RAID implementations done entirely in software, with no special hardware support

Hardware RAID: RAID implementations with special hardware

Use non-volatile RAM to record writes that are being executed

Beware: power failure during write can result in corrupted disk

E.g., failure after writing one block but before writing the second in a mirrored system

Such corrupted data must be detected when power is restored

Recovery from corruption is similar to recovery from failed disk

NV-RAM helps to efficiently detect potentially corrupted blocks

Otherwise all blocks of disk must be read and compared with mirror/
parity block

Hardware Issues (Cont.)

Latent failures: data successfully written earlier gets damaged
can result in data loss even if only one disk fails

Data scrubbing:

continually scan for latent failures, and recover from copy/parity

Hot swapping: replacement of disk while system is running, without power down

Supported by some hardware RAID systems,

reduces time to recovery, and improves availability greatly

Many systems maintain spare disks which are kept online, and used as replacements for failed disks immediately on detection of failure

Reduces time to recovery greatly

Many hardware RAID systems ensure that a single point of failure will not stop the functioning of the system by using

Redundant power supplies with battery backup

Multiple controllers and multiple interconnections to guard against controller/
interconnection failures

Authentication & Authorization

Authentication

The process of verifying the identity of a user or system.

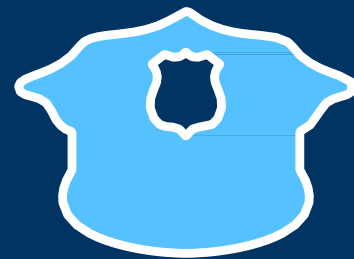
Authorization

Authorization is the process of determining whether an authenticated entity has permission to access a specific resource or perform a particular action

You are Wearing Many Hats



Many Hats You are Wearing



Database Administrator



Database
Layer



Application Developer



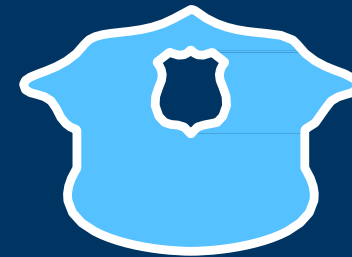
Application Layer



End
User

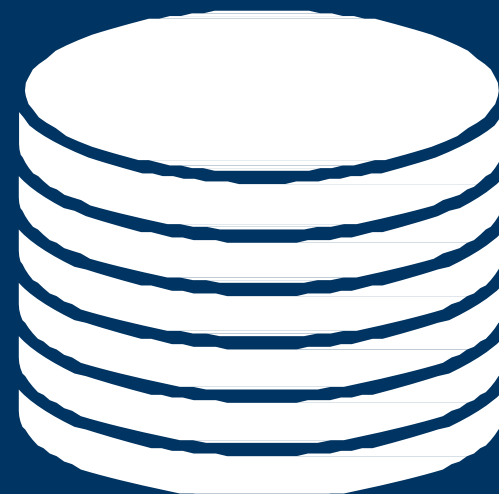


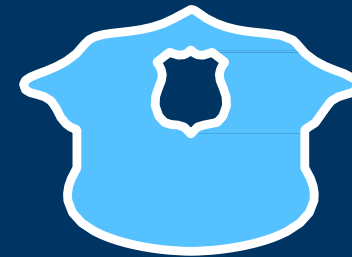
Presentation/View Layer



Database Administrator

Root





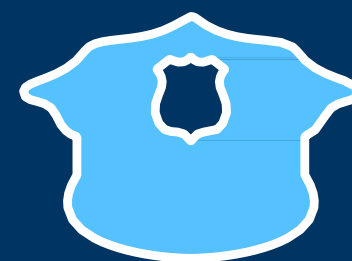
Database Administrator

Root

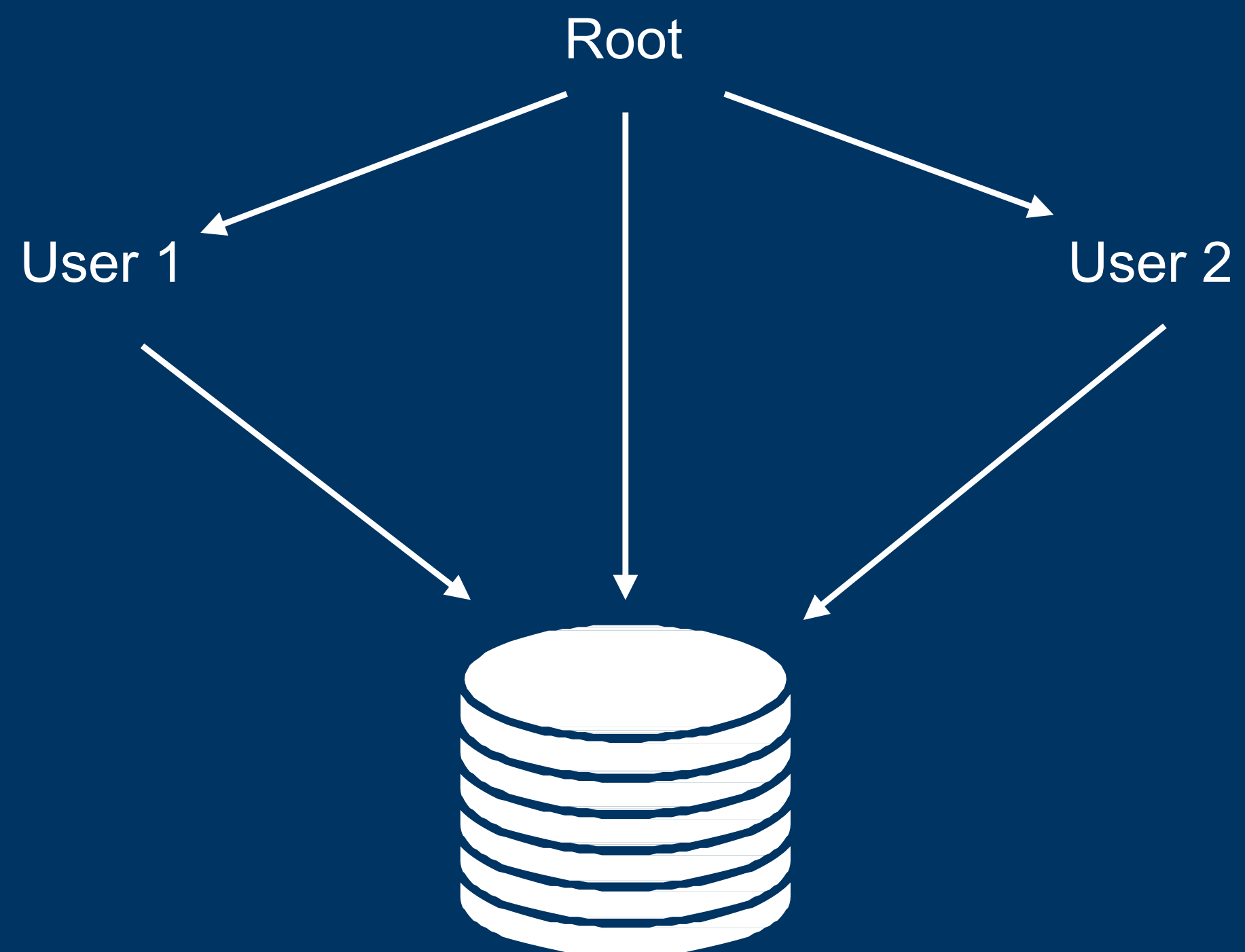
Full Access to all Databases and
Tables Create, Read, Update, Delete
(C.R.U.D)

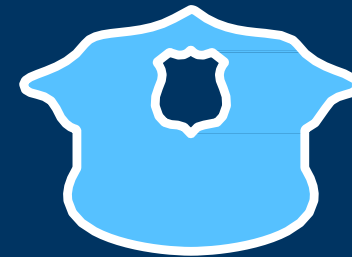
Each Type of Authorization is called a Privilege

1. Create, Read, Update, and Delete any user account
2. Grant and revoke privileges from other users.
3. Start, stop, and restart database server.
4. CRUD database schemas, tables
5. Manage security policies, encryption settings, and access controls



Database Administrator

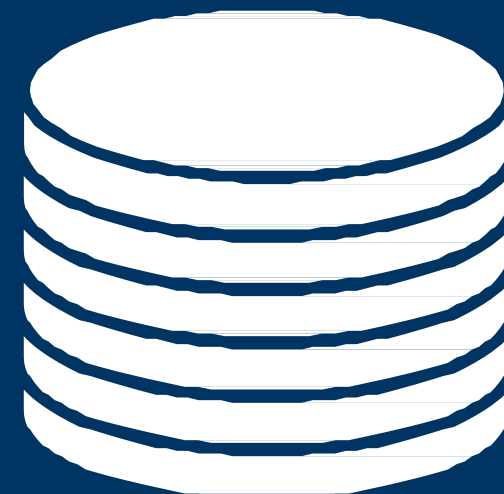


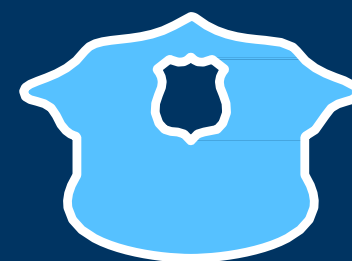


Database Administrator
CTO/ Head of Database
administration

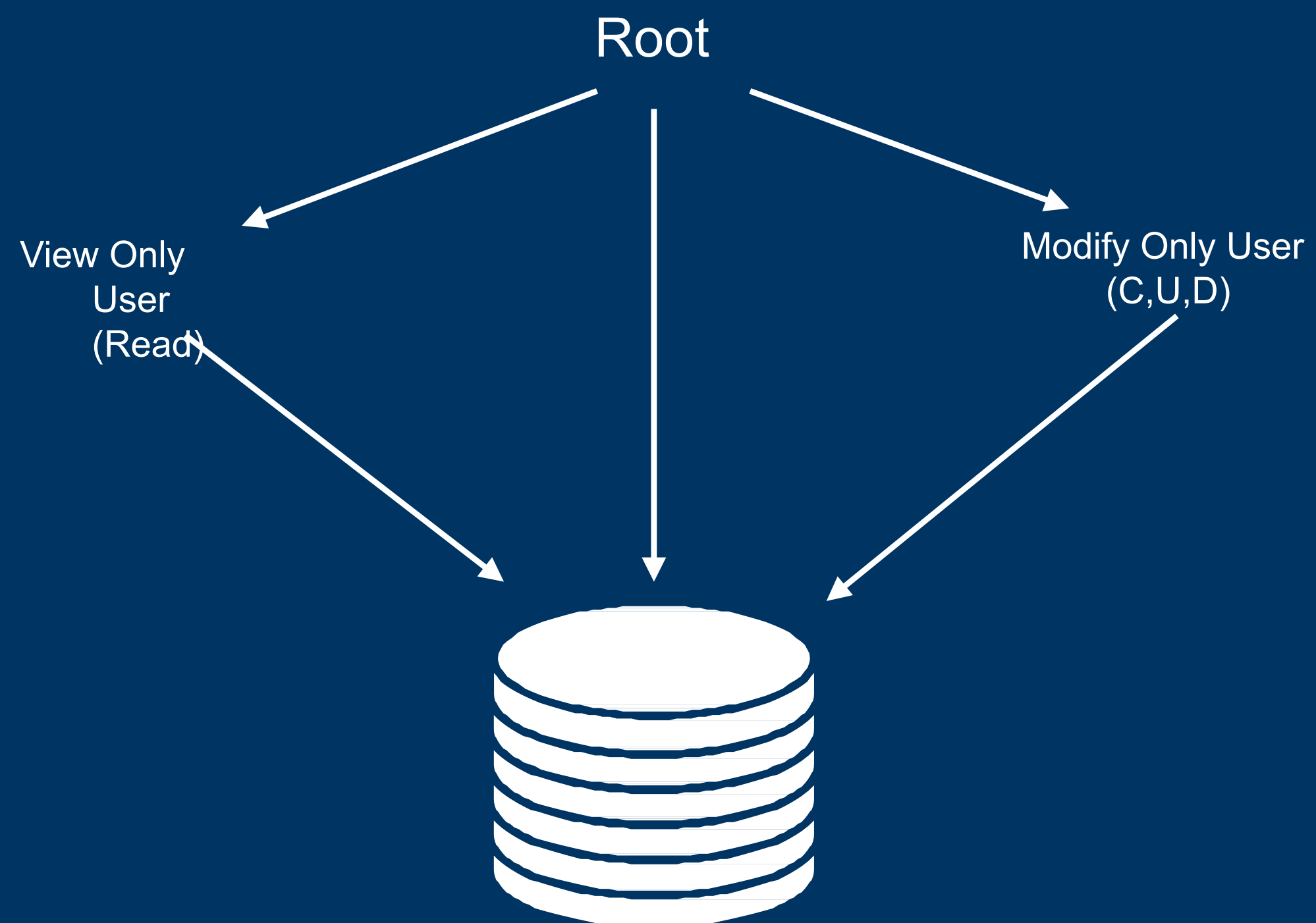
Sally (DB
Engineer)

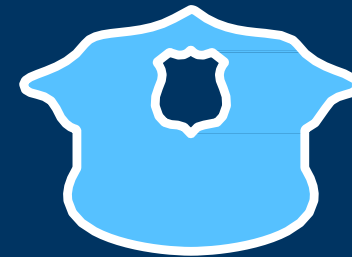
Joe (DB
Engineer)





Database Administrator

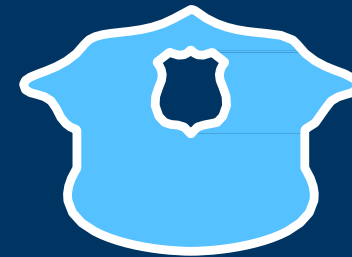




Database Administrator

Create
User

```
CREATE USER 'username'@'%' IDENTIFIED BY  
'password';
```



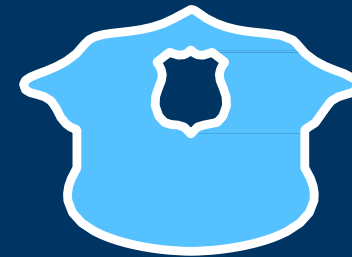
Database Administrator

Root

Key word Grant

grant <privilege list>
on <relation name or view name>
to <user/role list>;

```
GRANT EXECUTE ON PROCEDURE mydb.myprocedure TO  
myrole;
```



Database Administrator

Root

Key word Role

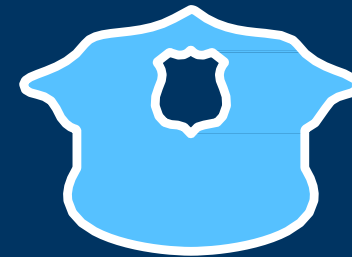
Create Role teacher;

Grant

SELECT On

grades

To teacher;



Database Administrator
Root

Key word Role

Create Role principal;

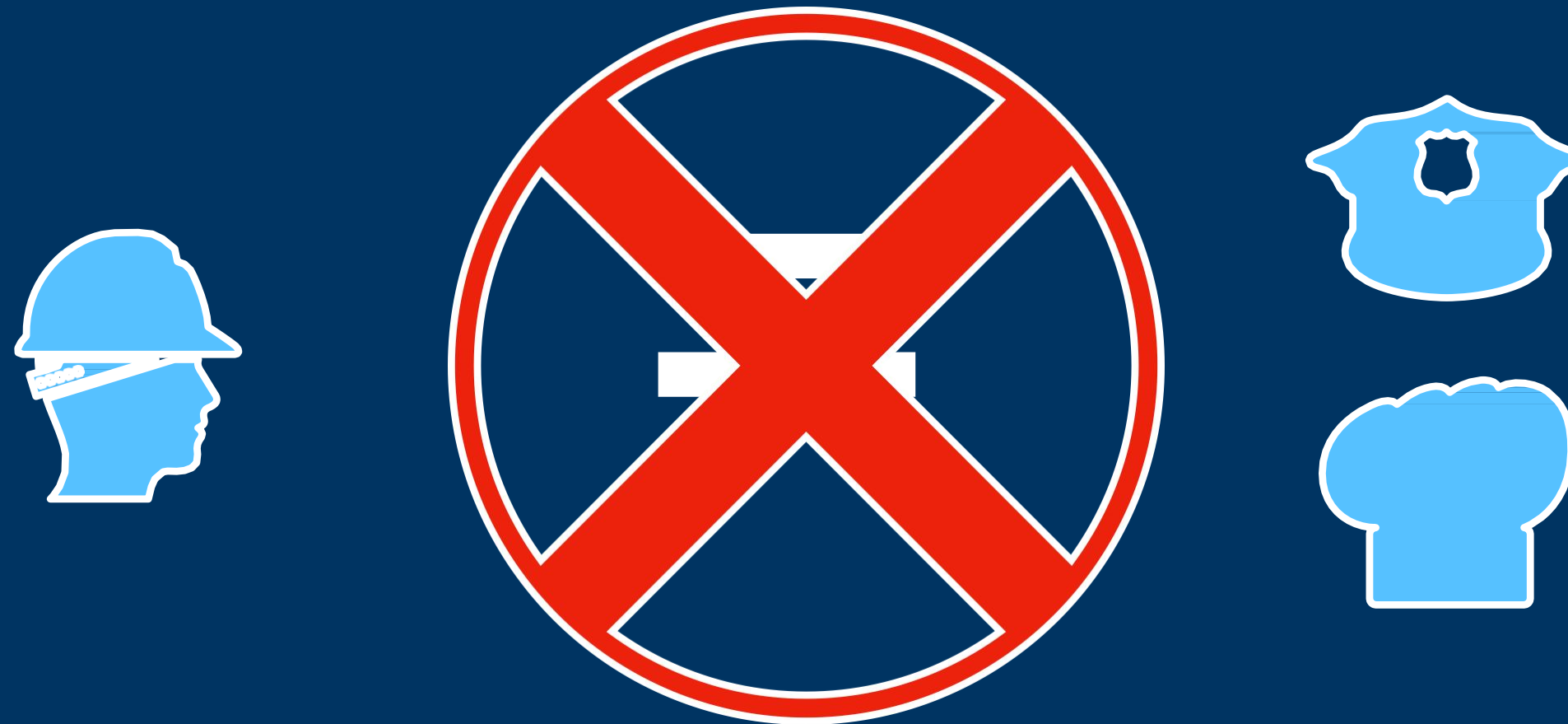
Grant teacher

To principal;

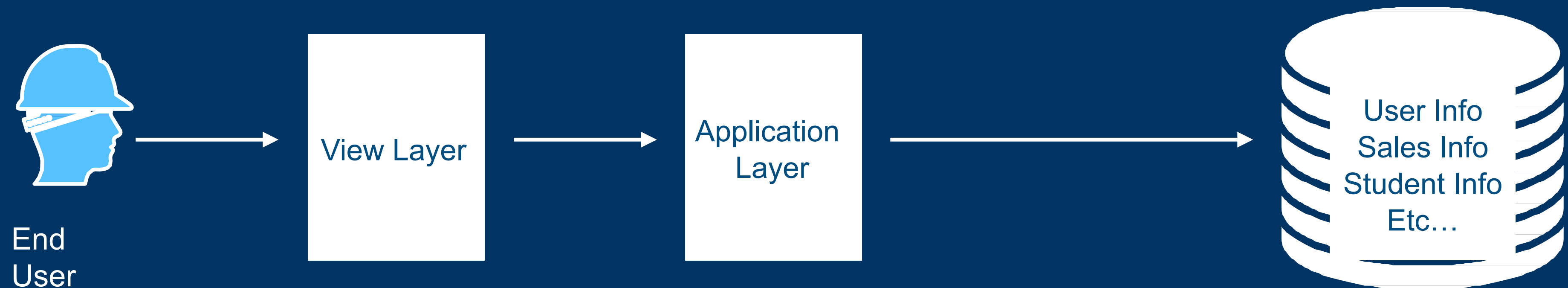
Users of the Database Vs. Users In the Database

Users of the Database Vs. Users In the Database

Is this the same User as the Database User?

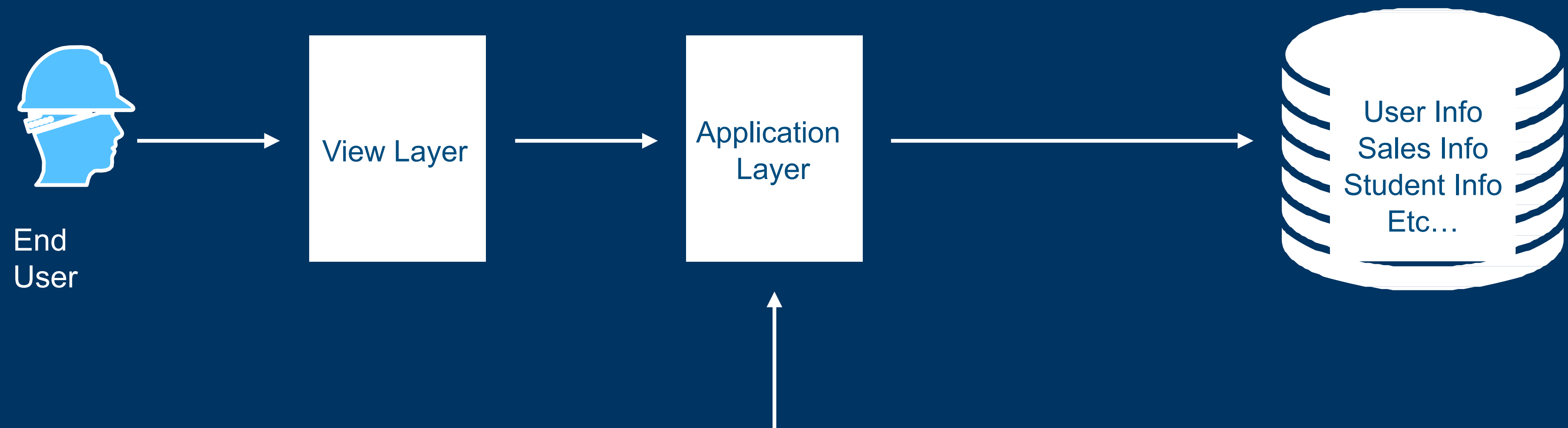


End User can access data in database but
cannot access database like User of
Database



Is the user who they say they are? —————> Authentication

Does the User have access to Make Changes or View? —————> Authorization



Where does this happen?



Create End User
(Application Or Database
Admin Creates)

```
Create Table users (  
  Id  
  email  
  password  
);
```

```
Create Roles(  
  ENUM('CEO, 'Sales  
  Manager', 'Sales  
  Rep'  
);
```



Database Definition
Language (DDL)

Note: This is only if the
application has users, like a
CRM.

View
Layer

Email

Password

Submit





Encrypt The Password

Use Encryption Package like BCrypt

Bcrypt

`pip install bcrypt`

1. Password - String provided by the User
2. Hash function - hashing algorithm provided by the package
3. Salt - randomly generated string that is provided to the algorithm

Bcrypt

```
pip install bcrypt
```

```
import bcrypt
```

```
password = "mypass12345!"
```

```
salt = bcrypt.gensalt( rounds=12 )
```

```
Store in the database → hashed_pw = bcrypt.hashpw( password, salt )
```


Bcrypt

```
pip install bcrypt
```

```
import bcrypt
```

Get From the Database → `hashed_pw =`
`$2a$12$eXNH1cC5mkIpJJZBdfEMZ.CuYZJ3P`
`Ucf/oZyxO4MdTGdB8/C6Cuwm`

```
password = "mypass12345!"
```

Boolean Value Returned → `bcrypt.checkpw = (password, hashed_pw)`